



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Uniform-Cost Search (UCS): An uninformed search algorithm that evaluates nodes solely based on the backward path cost from the start node:
 $f(n)=g(n)$. It expands nodes in uniform-cost concentric contours without any direction toward the goal location.

A* Search: An informed search algorithm that evaluates nodes by combining the actual backward path cost with an estimated forward heuristic cost to the goal:
 $f(n)=g(n)+h(n)$. This directs node expansion toward the goal, drastically reducing the number of explored nodes while maintaining optimality.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Why it is Admissible: A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal ($h(n) \leq h^*(n)$). On a 4-way grid (Up, Down, Left, Right) with unit cost per step, the absolute minimum number of steps between (x_1, y_1) and (x_2, y_2) in an obstacle-free grid is $|x_1 - x_2| + |y_1 - y_2|$. Any obstacles (walls) only increase the true shortest path length ($h^*(n) \geq h(n)$). Therefore, Manhattan distance is always $\leq h^*(n)$, guaranteeing admissibility.

Consequence of Non-Admissibility: If $h(n)$, $h(n)$ is not admissible (i.e., it overestimates the true cost), A^* loses its optimality guarantee. The search might overestimate the cost of the true optimal path and prematurely terminate with a suboptimal path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

Admissibility: No, Manhattan distance is no longer admissible for 8-way movement.

Why?

In 8-way movement, the agent can move from (0,0) to (1,1) in a single diagonal step. The actual cost depends on the cost model:

If diagonal cost = 1, the true cost is $h^*(n) = 1$.

If diagonal cost = $\sqrt{2} \approx 1.414$, the true cost is $h^*(n) = \sqrt{2}$.

However, Manhattan distance computes:

$$h_{\text{Manhattan}}(n) = |1-0| + |1-0| = 2$$

Since $2 > 1$ (or $2 > 1.414$), the heuristic overestimates the true cost, violating the admissibility condition ($h(n) \leq h^*(n)$ for all nodes).

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Here is the formatted and polished version of your heuristic description:

A Much Stronger & Admissible Heuristic: The MST Relaxation Heuristic

$$h(n) = \text{distance}(\text{agent}, \text{closest_food}) + \text{cost}(\text{MST}(\text{all_remaining_food}))$$

How It Works:

Distance to nearest food – Calculate the Manhattan distance from the agent's current position to the closest remaining food pellet.

MST over all food – Construct a Minimum Spanning Tree (MST) that connects all remaining uncollected food items, using pairwise Manhattan distances as edge weights.

Heuristic sum – Add the distance to the nearest food to the total edge cost of the MST.

Why It Is Stronger & Admissible:

Any valid path that collects all remaining food pellets must, at minimum, form a spanning subgraph that connects all those food locations.

The MST gives the absolute minimum possible cost to interconnect those points (ignoring obstacles), so it is a lower bound on the actual travel required between foods.

Adding the cost to reach the first food ensures the heuristic accounts for the full start-to-finish structure.

Thus, the true optimal path cost $h^*(n)$ can never be less than this heuristic value:

$$h(n) \leq h^*(n)$$

This makes $h(n)$ strictly admissible while providing a significantly tighter lower bound than simpler heuristics (e.g., just the nearest food distance).

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING